DSA 8070 R Session 14: Nonlinear Dimensionality Reduction and Manifold Learning

Whitney Huang, Clemson University

Contents

Recap: Linear PCA . . . 1
Polynomial PCA . . . 2
Idea . . . 2
Example: Noisy quadratic curve . . . 2
Kernel PCA . . . 6
Swiss Roll: Two Underlying Coordinates . . . 6
Gaussian Kernel and Bandwidth . . . 7
Compare the Representations . . . 8
Change the Bandwidth: Change the Representation . . . 9
Takeaway . . . 10
Nonlinear Manifold Learning . . . 10
Isomap: Preserve Distances Along the Surface . . . 10
LLE: Preserve Local Reconstruction Relationships . . . 11
t-SNE: Preserve Local Similarities . . . 12
UMAP: Preserve a Weighted Neighborhood Structure . . . 13
Takeaway . . . 14

Recap: Linear PCA

Principal Component Analysis (PCA) finds an orthogonal linear transformation of the data that:

- maximizes variance along successive components, and
- equivalently, minimizes reconstruction error in a low-dimensional linear subspace.

If X is an $n \times p$ data matrix (columns centered), PCA finds eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p$ and eigenvectors $v_1, \ldots, v_p$ of the covariance matrix. The first k eigenvectors span the best k-dimensional linear subspace for the data.

PCA finds the best linear subspace for reconstruction under squared Euclidean error. However, data can have a simple low-dimensional structure that is curved rather than flat.

There are several ways to address this limitation. We can construct nonlinear features and apply PCA to those features, or use a kernel to perform PCA in an implicit feature space. These methods optimize variance in the chosen feature space; they do not automatically recover the underlying manifold coordinates or minimize reconstruction error in the original variables.

Polynomial PCA

Idea

Polynomial PCA generalizes PCA by **explicitly expanding** the feature space with polynomial terms (quadratic, cubic, etc.), and then applying ordinary linear PCA to the expanded features.

For example, with two variables X_1, X_2, a quadratic expansion is

$$X' = (X_1, X_2, X_1^2, X_2^2, X_1X_2).$$

We then run PCA on X'. If the true relationship between X_1 and X_2 is approximately quadratic, the smallest eigenvalues of $\mathrm{Cov}(X')$ correspond to directions representing the nonlinear constraint.

Example: Noisy quadratic curve

We simulate bivariate data lying near a quadratic curve, then compare linear PCA and quadratic PCA.

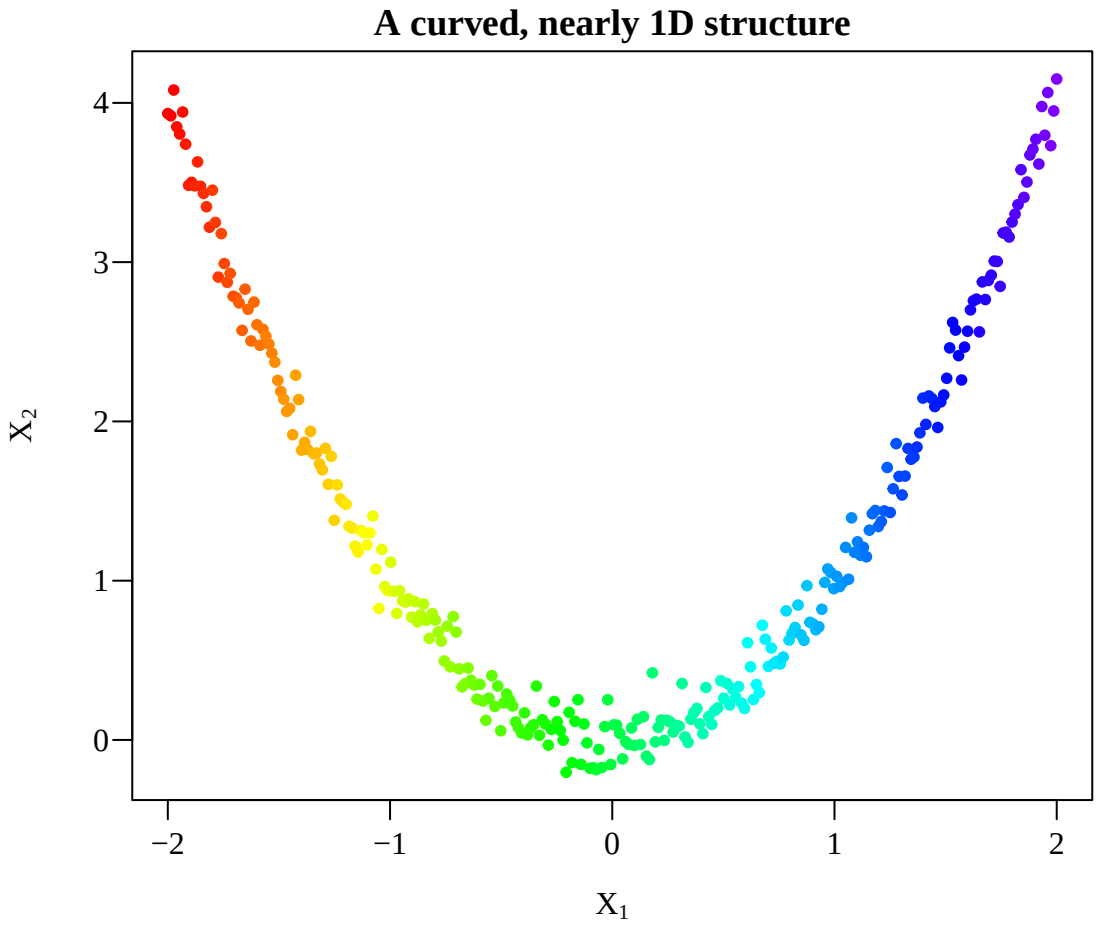
```
set.seed(123)

# Symmetric design makes the nonlinear relationship easy to see.
t <- seq(-2, 2, length.out = 300)
dat <- data.frame(x1 = t, x2 = t^2 + rnorm(length(t), sd = 0.12))

cols <- rainbow(100, start = 0, end = 0.75)[
  cut(t, breaks = 100, labels = FALSE)]

par(mar = c(4, 4, 1.5, 1), mgp = c(2.3, 0.6, 0), las = 1, family = "serif")

plot(dat$x1, dat$x2, col = cols, pch = 20,
     xlab = expression(X[1]), ylab = expression(X[2]),
     main = "A curved, nearly 1D structure")
```




```
pca_lin <- prcomp(dat, center = TRUE, scale. = FALSE)
summary(pca_lin)
```

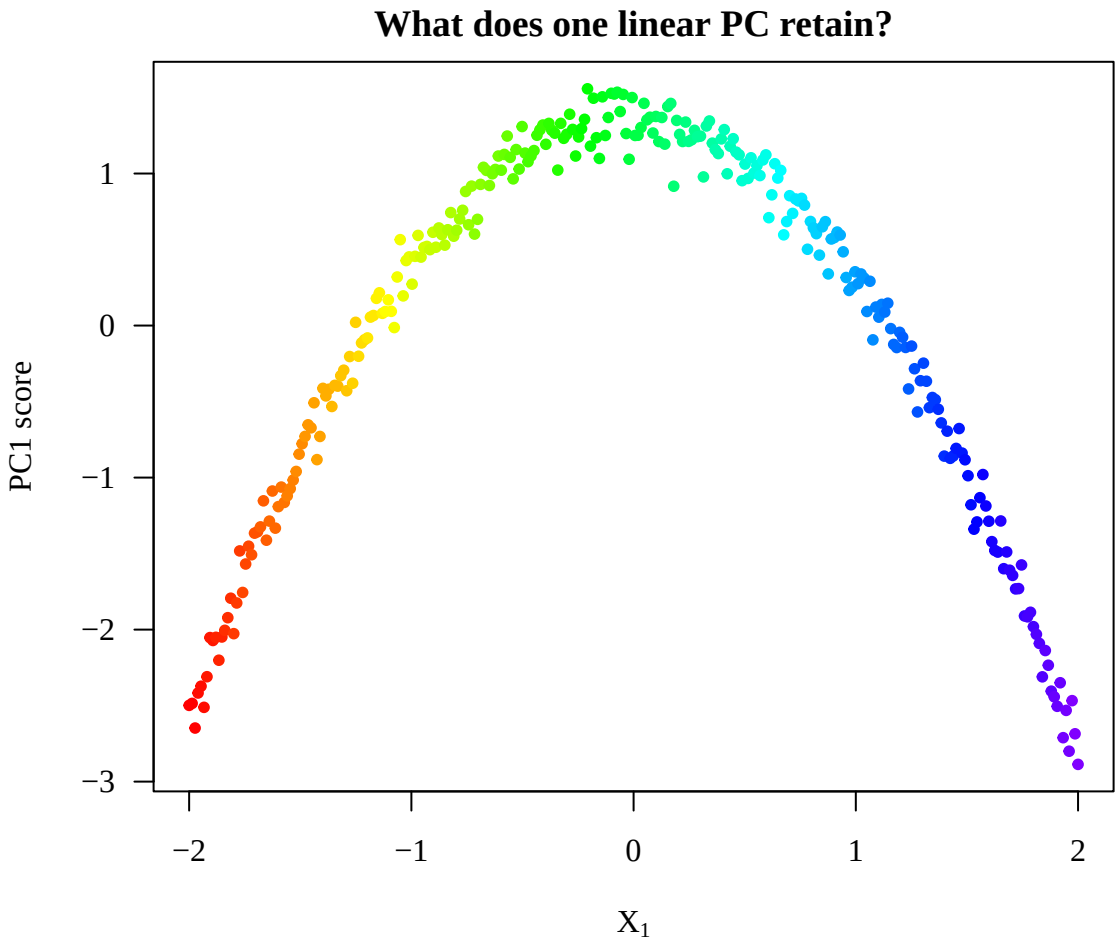
Linear PCA

```
## Importance of components:
##                           PC1    PC2
## Standard deviation     1.2184 1.1604
## Proportion of Variance 0.5244 0.4756
## Cumulative Proportion  0.5244 1.0000
```

```
pca_lin$rotation
```

```
##           PC1        PC2
## x1 -0.0427495 -0.9990858
## x2 -0.9990858  0.0427495
```

```
par(mfrow = c(1, 1), mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(dat$x1, pca_lin$x[, 1], col = cols, pch = 20, xlab = expression(X[1]), ylab = "PC1 score", main =
```



Linear

PCA summarizes each observation using a linear combination of the centered variables:

$$\text{PC1} = a(X_1 - \bar{X}_1) + b(X_2 - \bar{X}_2).$$

For our symmetric quadratic example, $X_2 \approx X_1^2$, the first principal component mainly reflects variation in X_2. Consequently, observations on opposite sides of the parabola can have similar PC1 scores, making them difficult to distinguish using PC1 alone.

Although the data lie close to a one-dimensional curve, **one linear principal component does not adequately describe position along that curve**. Keeping both PCs preserves the data but does not reduce the dimension. This motivates using nonlinear features to represent the quadratic relationship.

```r
# Add the feature needed to represent the quadratic relationship.
X_quad <- with(dat, cbind(x1 = x1, x2 = x2, x1_sq = x1^2))
pca_quad <- prcomp(X_quad, center = TRUE, scale. = FALSE)

# The last PC identifies a combination with very little variation.
v <- pca_quad$rotation[, 3]
round(v / v["x2"], 3)
```

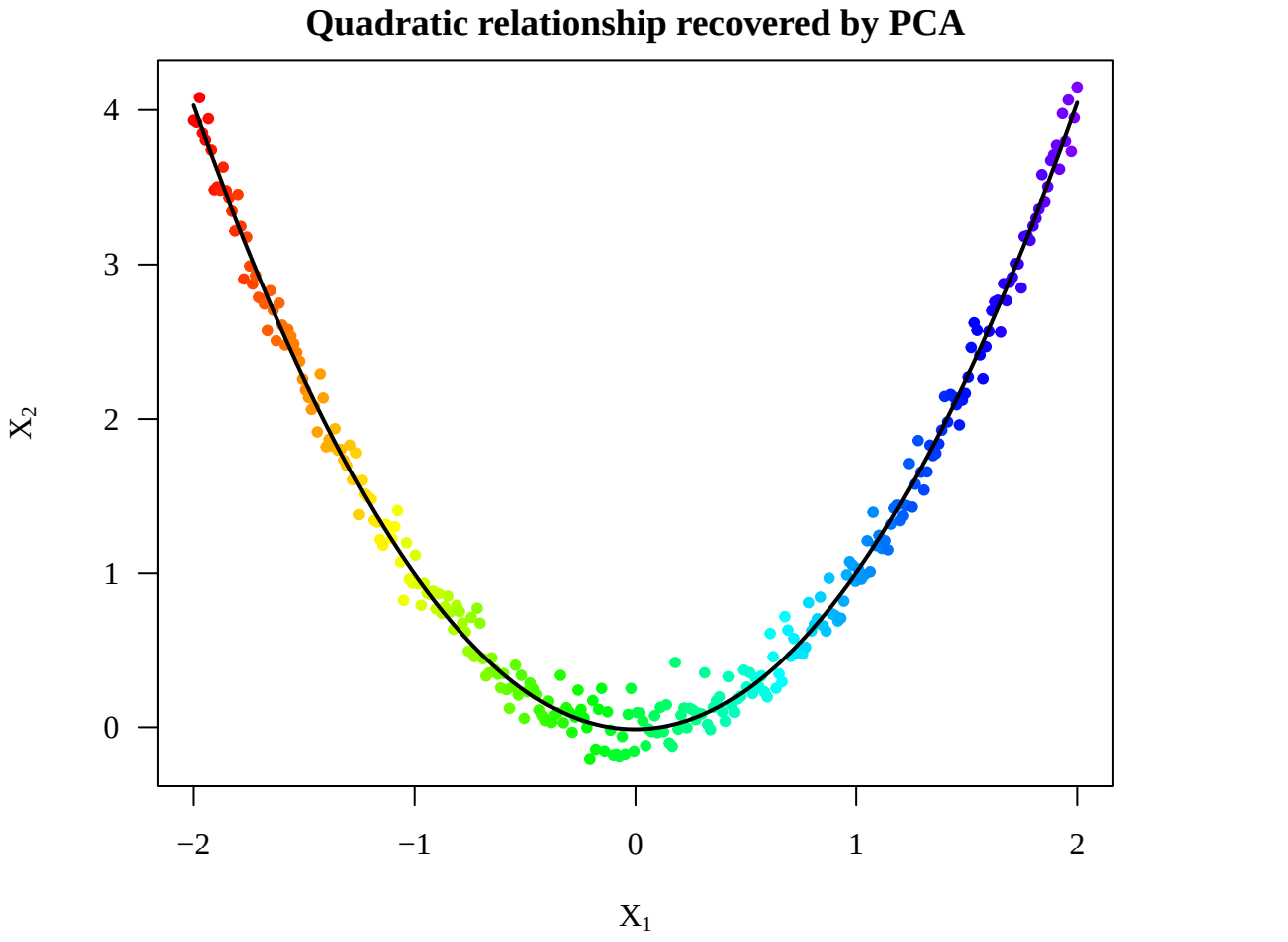
Quadratic PCA

```
##     x1     x2  x1_sq
## -0.004  1.000 -1.013
```

```r
# Express the constraint as x2 = b0 + b1*x1 + b2*x1^2.
b0 <- sum(v * pca_quad$center) / v["x2"]
b1 <- -v["x1"] / v["x2"]
b2 <- -v["x1_sq"] / v["x2"]
round(c(intercept = b0, linear = unname(b1), quadratic = unname(b2)), 3)
```

```
## intercept.x2       linear    quadratic
##       -0.014        0.004        1.013
```

```r
# Show the quadratic curve recovered from the PCA constraint.
x_grid <- seq(min(dat$x1), max(dat$x1), length.out = 200)
x2_fit <- b0 + b1 * x_grid + b2 * x_grid^2
par(mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(dat$x1, dat$x2, col = cols, pch = 20, xlab = expression(X[1]), ylab = expression(X[2]),
     main = "Quadratic relationship recovered by PCA")
lines(x_grid, x2_fit, lwd = 2)
```



We add X_1^2 as a new feature and apply PCA to (X_1, X_2, X_1^2). For our example, $X_2 \approx X_1^2$, so the combination $X_2 - X_1^2$ varies very little.

PCA detects this relationship through its **last component**, which has the smallest variance:

$$v_1(X_1 - \bar{X}_1) + v_2(X_2 - \bar{X}_2) + v_3(X_1^2 - \overline{X_1^2}) \approx 0.$$

Rearranging gives a quadratic curve in the original variables:

$$X_2 \approx b_0 + b_1 X_1 + b_2 X_1^2.$$

The black curve shows this relationship recovered from the PCA loadings. **Adding a nonlinear feature allows PCA to detect the quadratic constraint as a linear relationship in the expanded space.** Here we use the last PC to identify the constraint; we are not claiming that the first PC alone describes position along the curve.

Kernel PCA

Polynomial PCA constructs nonlinear features explicitly. Kernel PCA instead uses a kernel to compute inner products in an implicit feature space:

$$K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle.$$

It then finds directions of maximum variance in that feature space. **The resulting coordinates can be nonlinear functions of the original variables.**

We use the Swiss roll to examine what this changes relative to linear PCA. Our goal is to investigate the nonlinear representation, rather than assume that kernel PCA will unfold the roll.

```
library(kernlab)
library(scatterplot3d)
```

Swiss Roll: Two Underlying Coordinates

The observations are generated from two coordinates, t and h:

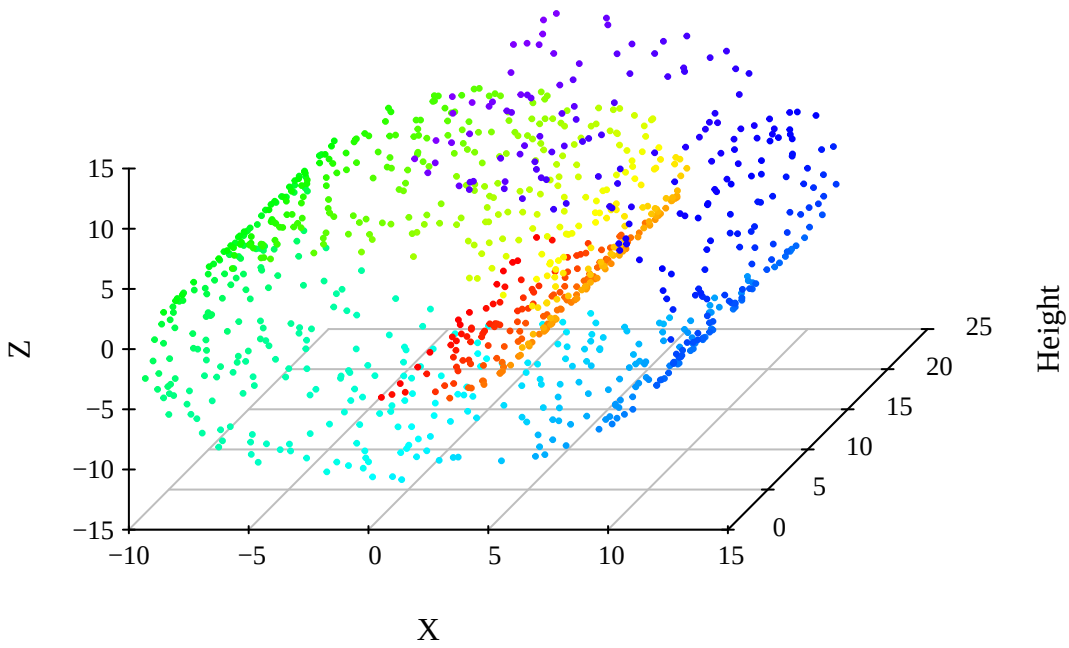
$$x = t\cos(t), \qquad y = h, \qquad z = t\sin(t).$$

Here, t determines position along the spiral and h determines height. Thus, the data form a two-dimensional curved surface in three-dimensional space.

We color points by t. Following the colors lets us track position along the roll in every plot. Neither t nor h is supplied separately to PCA.

```
set.seed(123)
n <- 1000
t <- 3 * pi / 2 * (1 + 2 * runif(n))
h <- 21 * runif(n)
swiss_roll <- data.frame(x = t * cos(t), y = h, z = t * sin(t))
X3 <- as.matrix(swiss_roll)
cols <- rainbow(100, start = 0, end = 0.75)[cut(t, breaks = 100, labels = FALSE)]

par(mar = c(4, 4, 2, 1), las = 1, family = "serif")
scatterplot3d(X3[, 1], X3[, 2], X3[, 3], color = cols, pch = 20, cex.symbols = 0.5, angle = 60, xlab =
```



The color sequence follows the curved surface. Points on different turns can be close in three-dimensional Euclidean distance even though they are far apart along the surface.

Gaussian Kernel and Bandwidth

We use the Gaussian kernel from the slides:

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right).$$

Nearby observations have similarity close to one; distant observations have similarity closer to zero. The bandwidth σ determines the distance over which similarity decreases.

In `kernlab`, the argument named `sigma` uses a different convention:

$$K(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|^2), \qquad \gamma = \frac{1}{2\sigma^2}.$$

We therefore use `bandwidth` for the slide parameter and convert it to `gamma` when calling the software.

```
# Use a typical pairwise distance as an initial bandwidth.
bandwidth <- median(dist(X3))
gamma <- 1 / (2 * bandwidth^2)

# Linear PCA in the original space.
pca_lin_3d <- prcomp(X3, center = TRUE, scale. = FALSE)
lin_scores <- pca_lin_3d$x[, 1:2]
```

```r
# Kernel PCA in the implicit feature space.
kpc <- kpca(X3, kernel = "rbfdot", kpar = list(sigma = gamma), features = 2)
pc_scores <- rotated(kpc)
```

As in the slides, kernel PCA constructs the Gram matrix K, centers it in feature space, and computes its eigenvectors:

$$\widetilde{K} = HKH, \qquad H = I_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\mathsf{T}.$$

For a unit eigenvector v_ℓ with eigenvalue μ_ℓ, the corresponding score vector is

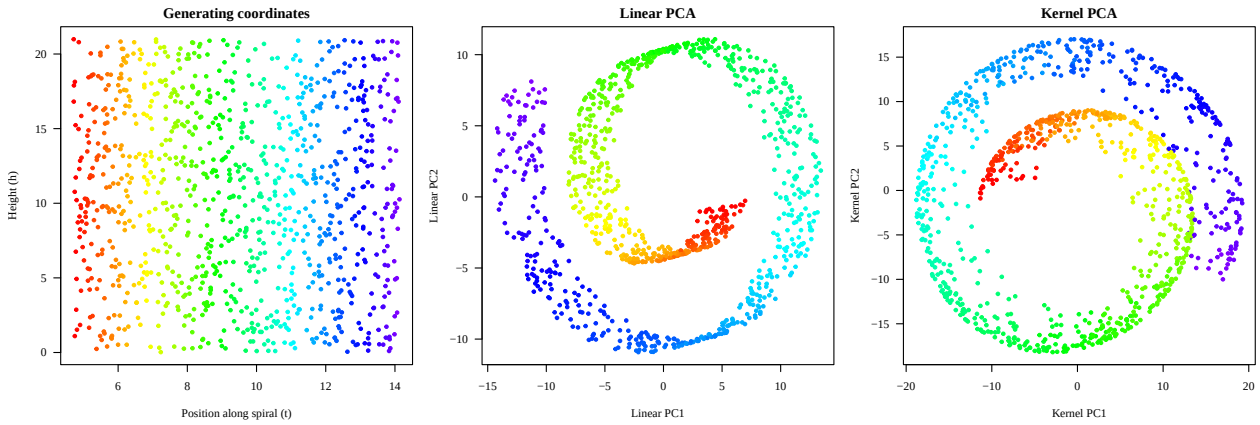
$$z_\ell = \sqrt{\mu_\ell}\, v_\ell.$$

The function `kpca()` handles kernel construction, centering, and eigendecomposition. `rotated()` extracts its training scores; their overall normalization can differ from the slide convention without changing the plotted structure.

Compare the Representations

The first panel shows the generating coordinates (t, h) as a reference. The other panels show representations computed using only the observed variables (x, y, z).

```r
par(mfrow = c(1, 3), mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(t, h, col = cols, pch = 20, xlab = "Position along spiral (t)", ylab = "Height (h)", main = "Genera
plot(lin_scores, col = cols, pch = 20, xlab = "Linear PC1", ylab = "Linear PC2", main = "Linear PCA")
plot(pc_scores, col = cols, pch = 20, xlab = "Kernel PC1", ylab = "Kernel PC2", main = "Kernel PCA")
```




```r
par(mfrow = c(1, 1))
```

Linear PCA projects the data onto a plane. Because the surface is rolled, distinct portions can overlap in this projection.

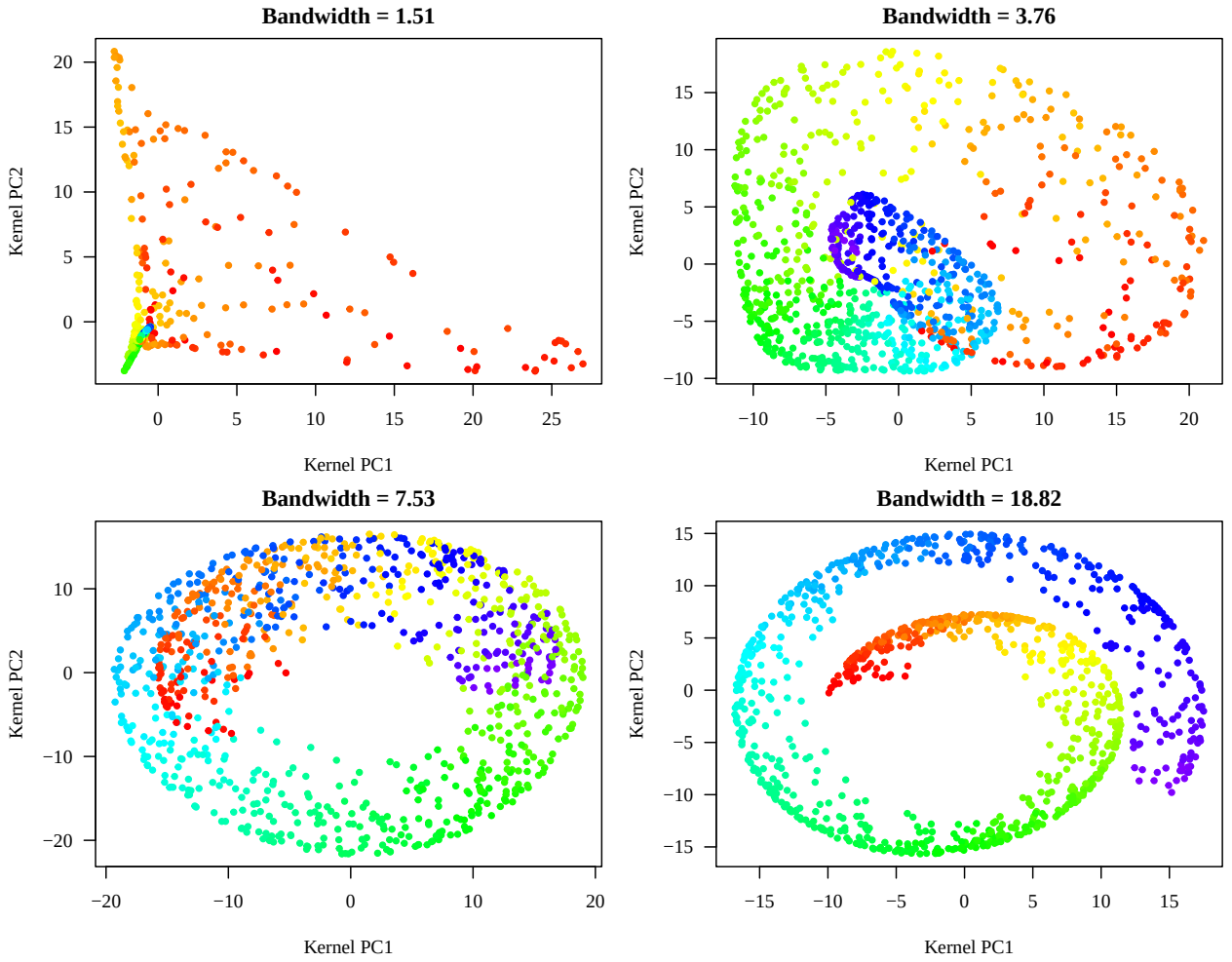
Kernel PCA constructs nonlinear coordinates using the chosen similarities. Its plot may rearrange the observations, but a different-looking plot does not by itself demonstrate successful recovery of the surface.

Use the colors to assess the result: Do they progress smoothly across the embedding, or do different portions of the roll remain mixed? The generating coordinates provide a reference for ordering, but (t, h) is not a distance-preserving flattening because t measures spiral angle rather than arc length.

Change the Bandwidth: Change the Representation

The initial bandwidth is a starting choice, not an optimal value. We now repeat kernel PCA using smaller and larger bandwidths while keeping the data and colors fixed.

```
bandwidths <- bandwidth * c(0.1, 0.25, 0.5, 1.25)
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1), las = 1, family = "serif")
for (bw in bandwidths) {
  fit <- kpca(X3, kernel = "rbfdot", kpar = list(sigma = 1 / (2 * bw^2)), features = 2)
  scores <- rotated(fit)
  plot(scores, col = cols, pch = 20, xlab = "Kernel PC1", ylab = "Kernel PC2", main = paste("Bandwidth =
}
```




```
par(mfrow = c(1, 1))
```

A smaller bandwidth emphasizes similarities over shorter distances. A larger bandwidth makes similarity decrease more gradually with distance. Changing the bandwidth therefore changes the feature space and the variation captured by the leading components.

Compare the color ordering and overlap across panels. Axis signs may reverse, so a mirrored plot alone does not indicate a meaningful difference.

Kernel PCA does not explicitly preserve distances along the surface. Its RBF similarities are based on Euclidean distances, which can connect different turns of the roll. Consequently, kernel PCA can extract nonlinear features without successfully unfolding the manifold.

Takeaway

Linear PCA captures variance in the original variables. Kernel PCA captures variance in a kernel-defined feature space, producing nonlinear coordinates without explicitly constructing the features.

The Swiss roll illustrates both the flexibility and the limitation: **the representation depends on bandwidth, and nonlinear feature extraction does not guarantee manifold unfolding.** This motivates methods such as Isomap, which explicitly uses approximate distances along the manifold.

Nonlinear Manifold Learning

We continue with the Swiss roll, comparing methods that preserve different aspects of its structure:

- **Isomap:** approximate distances along the surface.
- **LLE:** local reconstruction relationships.
- **t-SNE and UMAP:** neighborhood similarities, primarily for visualization.

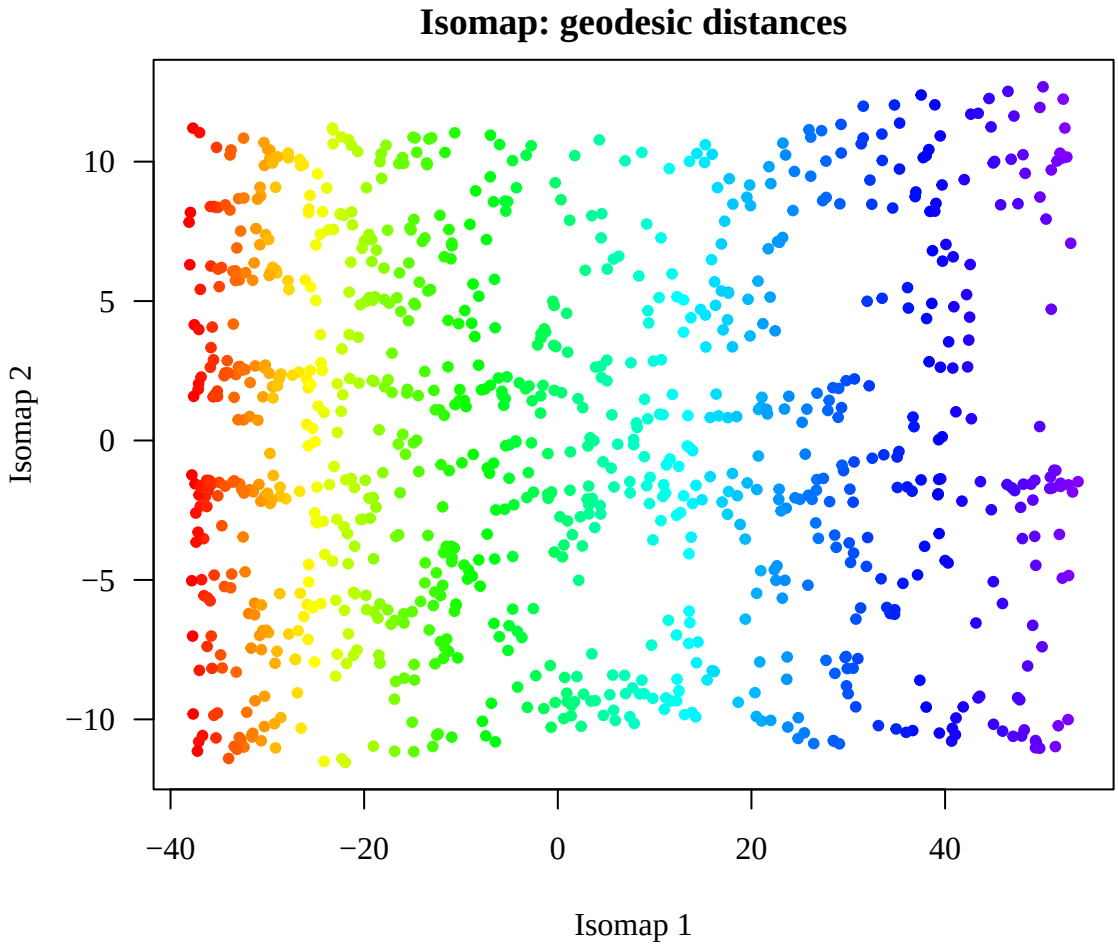
All methods use only (x, y, z). Colors indicate position along the roll, t, and help us assess the embeddings; they are not used for fitting.

```
X <- as.matrix(swiss_roll[, c("x", "y", "z")])
cols <- rainbow(100, start = 0, end = 0.75)[cut(t, breaks = 100, labels = FALSE)]
```

Isomap: Preserve Distances Along the Surface

Isomap connects nearby observations, approximates geodesic distances using shortest paths through this neighborhood graph, and applies classical multidimensional scaling to those distances.

```
library(dimRed)
swiss_dr <- as(as.data.frame(X), "dimRedData")
iso_emb <- dimRed::embed(swiss_dr, "Isomap", knn = 10, ndim = 2)
iso_coords <- getData(getDimRedData(iso_emb))
par(mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(iso_coords, col = cols, pch = 20, xlab = "Isomap 1", ylab = "Isomap 2", main = "Isomap: geodesic di
```



Look for a smooth color progression with different turns separated. The neighborhood size matters: too few neighbors can disconnect the graph; too many can create shortcuts between turns.

LLE: Preserve Local Reconstruction Relationships

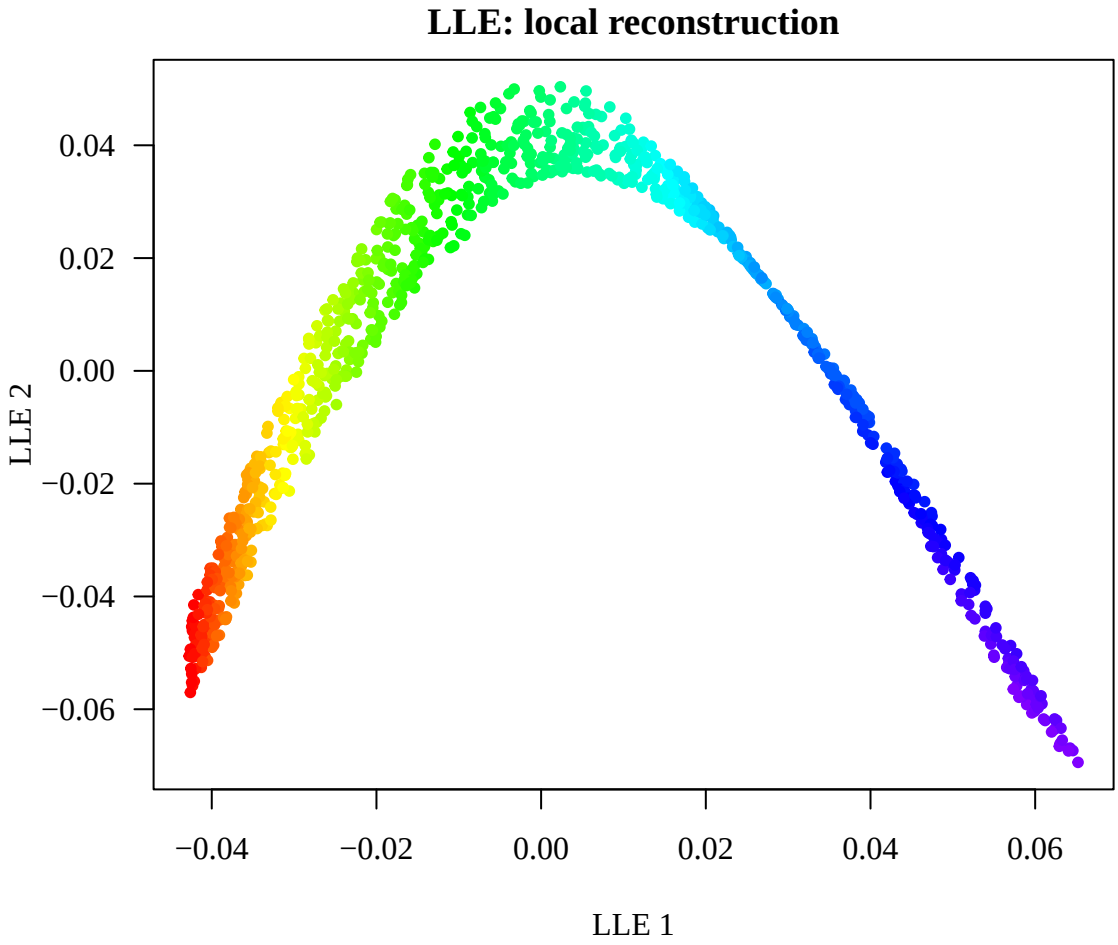
LLE first expresses each observation as a weighted combination of its neighbors. It then finds low-dimensional coordinates that preserve those reconstruction weights.

```
library(Rdimtools)
```

```
## Warning in rgl.init(initValue, onlyNULL): RGL: unable to open X11 display
```

```
## Warning: 'rgl.init' failed, will use the null device.
## See '?rgl.useNULL' for ways to avoid this warning.
```

```
set.seed(123)
out_lle <- Rdimtools::do.lle(X, ndim = 2, type = c("knn", 12), symmetric = "union",
                             weight = TRUE, preprocess = "null", regtype = TRUE, regparam = 1)
par(mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(out_lle$Y, col = cols, pch = 20, xlab = "LLE 1", ylab = "LLE 2", main = "LLE: local reconstruction
```

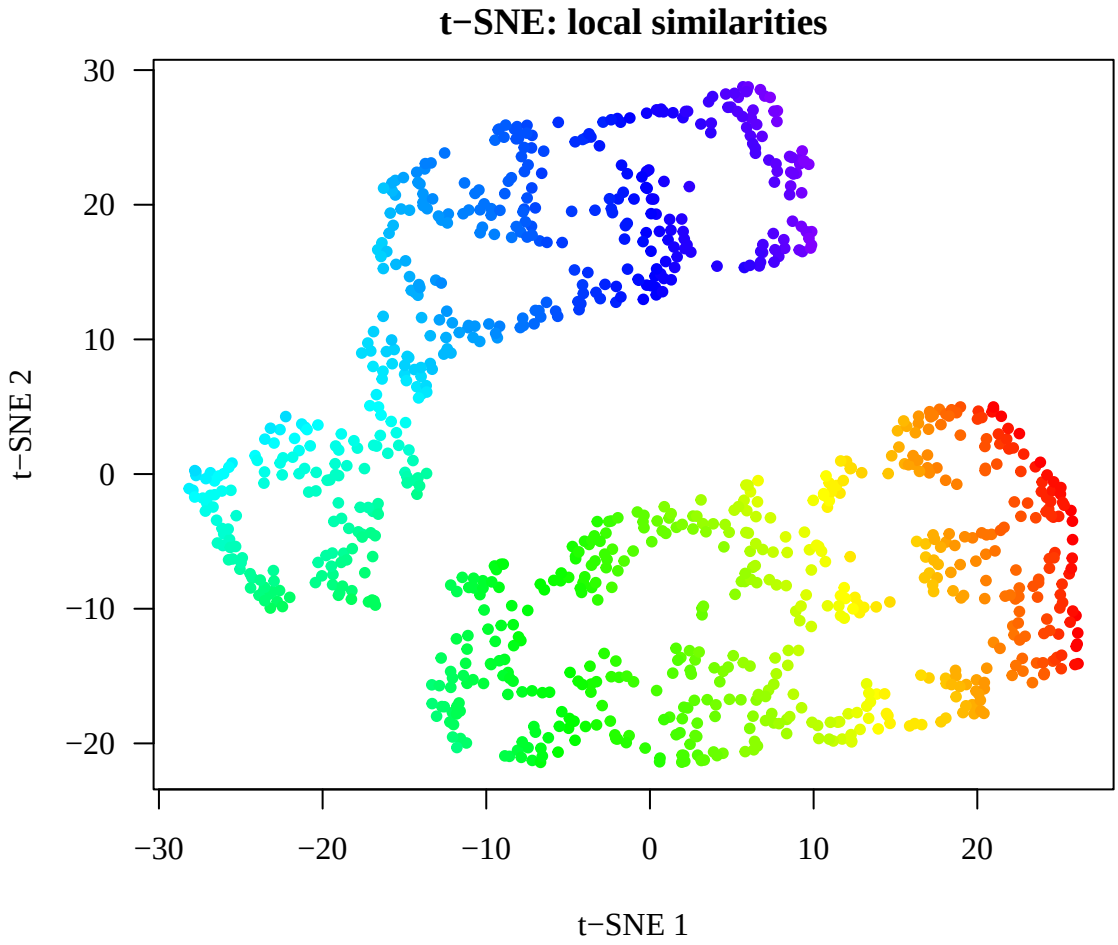


Examine whether nearby colors remain together. Unlike Isomap, LLE does not explicitly preserve global geodesic distances. Its result depends on neighborhood selection and the stability of the reconstruction weights.

t-SNE: Preserve Local Similarities

t-SNE represents pairwise similarities as probabilities and minimizes the Kullback–Leibler divergence between the original and embedded similarity distributions.

```
library(Rtsne)
set.seed(123)
tsne_result <- Rtsne::Rtsne(X, dims = 2, perplexity = 30, verbose = FALSE, check_duplicates = TRUE)
par(mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(tsne_result$Y, col = cols, pch = 20, xlab = "t-SNE 1", ylab = "t-SNE 2", main = "t-SNE: local simil
```

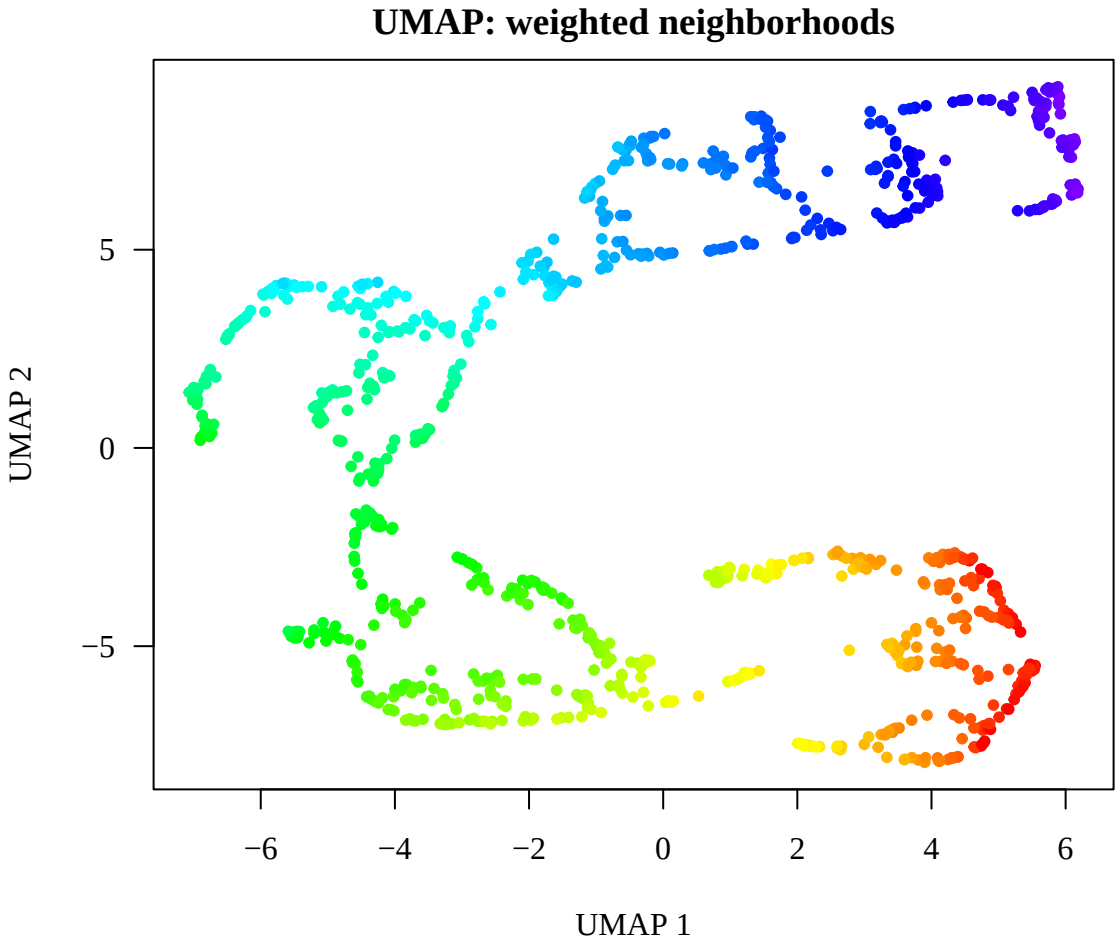


Nearby points often represent similar observations. However, apparent cluster sizes, distances between groups, and axis directions should not be interpreted literally. Separate-looking groups need not indicate genuine clusters: the Swiss roll is a continuous surface.

UMAP: Preserve a Weighted Neighborhood Structure

UMAP constructs a weighted neighborhood graph and finds a low-dimensional embedding that approximately preserves those relationships.

```
library(umap)
set.seed(123)
umap_result <- umap::umap(X, n_neighbors = 15, min_dist = 0.1)
par(mar = c(4, 4, 2, 1), las = 1, family = "serif")
plot(umap_result$layout, col = cols, pch = 20, xlab = "UMAP 1", ylab = "UMAP 2", main = "UMAP: weighted
```



Takeaway

Method	Main objective
Kernel PCA	Capture variance in an implicit feature space
Isomap	Preserve approximate geodesic distances
LLE	Preserve local reconstruction weights
t-SNE	Preserve local similarity probabilities
UMAP	Preserve weighted neighborhood relationships

Choose the method according to the structure you want to preserve. A visually attractive embedding alone does not establish that the underlying geometry has been recovered.